

Predicting Difficulty and Algorithmic Tags of Competitive Programming Problems

Three Approaches: Sentence Embeddings, Graph Multi-Graph Ensembles, and
LLM-Augmented Pipelines

Preda Marius Anghel Dragos Dascalu Stefan Pietraru Alfred

May 27, 2026

Abstract

We classify competitive programming problems by difficulty (8-way A–H) and algorithmic tags (multi-label) from their natural-language description and one or more C++ solutions. We compare three independent approaches: (i) a sentence-embedding plus MLP classifier on top of MiniLM and mpnet encoders; (ii) a graph-based classifier that parses every available C++ solution into an AST graph and aggregates predictions across the multi-graph ensemble; and (iii) an LLM-based pipeline built on the OpenAI chat and embedding APIs, augmented with prompt training, decomposed prompts with self-consistency, per-tag threshold tuning, post-hoc class-prior calibration, a supervised classifier on OpenAI embeddings, engineered code features, fixed-weight ensembling, and a stacked meta-classifier. We report results per approach and a final cross-approach comparison. The graph pipeline benefits dramatically from data refinement plus multi-graph aggregation; the LLM pipeline delivers the strongest tag F1 with the lowest engineering cost; the sentence-embedding baseline remains competitive on strict difficulty accuracy.

1 Introduction

Programming problem statements carry rich algorithmic signal that is often easier to extract than the contest-style difficulty letter assigned to them. Our goal is to train a classifier that, given a problem description and an example C++ solution, predicts:

- **Difficulty** on a fine-grained eight-level scale A–H,
- **Algorithmic tags** from a closed vocabulary of 35 tags.

The dataset consists of approximately 1,900 labeled problems split into train (about 1,338), validation (286), and test (288) sets using a deterministic stratified shuffle.

2 Evolution Across Milestones

The project evolved from a code-graph prototype into a three-branch research pipeline: graph neural networks over C++ program structure, classical sentence-embedding classifiers over text/code, and LLM-based semantic classifiers. This section summarizes that trajectory and explains why each later milestone changed direction.

2.1 Milestone 1: Code Property Graph Hypothesis

The first milestone established the original research hypothesis: raw ASTs contain too much syntactic noise for algorithmic prediction, but Code Property Graphs (CPGs) can expose the data-flow and structural relations that matter for competitive-programming algorithms. The milestone implemented a complete end-to-end beta pipeline:

- **Data curation.** Raw Codeforces problems from the Description2Code dataset were filtered to valid C++ solutions with at least one tag. Non-standard difficulty labels such as A1/B2 and classes with fewer than 10 examples were pruned.
- **Graph extraction.** Tree-sitter parsed C++ submissions; a custom extractor converted the syntax tree into typed nodes (`Variable`, `Loop`, `Operation`, `DataStructure`) and semantic edges (`READS`, `WRITES`, `NESTED_IN`, `CALLS`).
- **GNN training.** Graphs were serialized as PyTorch Geometric `HeteroData`. An RGCN backbone encoded relation-specific messages and fed two prediction heads: cross-entropy for difficulty and BCE-with-logits for tags.
- **Evaluation.** Difficulty accuracy and per-tag F1 were computed on a held-out test split.

Milestone 1 also introduced the first major extractor ablation. V1 extracted only loops and declarations; V2 added data-flow and operation edges; V3 added array detection, first-class STL operation capture, and “hairball prevention” by ignoring loop iterator variables such as `i`, `j`, `k`, `n`, and `m`. This progression mattered: the milestone reported tag macro-F1 improvements from V1 GraphSAGE (0.11) to V3 GraphSAGE (0.23), then to V3 RGCN with max+mean pooling (0.29).

The best early graph model reached roughly 32–33% 8-way difficulty accuracy (about $2.6\times$ random chance) and strong F1 on structurally visible tags such as *implementation*, *data structures*, *DFS*, and *DP*. The same report identified the central limitation: math-heavy tags such as *number theory*, *geometry*, and *binary search* are often defined by the problem statement’s reasoning rather than by surface AST shape. That observation motivated later multimodal and LLM directions.

2.2 Milestone 2: Text and Embedding Baselines

The next phase broadened the project beyond graphs. Instead of relying only on C++ structure, the project introduced text-centric baselines that combine the problem statement and reference solution. Sentence-transformer encoders such as `all-MiniLM-L6-v2` and `all-mpnet-base-v2` provided dense representations, and a shallow multi-task classifier predicted difficulty and tags.

This phase answered a key question: how much can be learned without explicit graph construction? The answer was mixed. Solution-only input produced the strongest strict difficulty accuracy among the recorded sentence-embedding runs (0.397), while augmented and merged-label settings improved macro-F1 and tag micro-F1. However, the baseline exposed two limitations:

- context length was capped by the encoder at 512 tokens, so requested 1024/2048-token experiments did not change the effective input;
- exact multi-label tag prediction remained extremely hard, with subset accuracy near zero.

This phase made the project more robust experimentally: it gave a non-graph baseline, clarified the impact of input mode (description-only vs. solution-only vs. combined), and showed that strict A–H difficulty accuracy is noisy even when the representation changes.

2.3 Milestone 3: LLM and Multi-Graph Consolidation

The current milestone turned the earlier limitations into two complementary improvements.

First, the LLM pipeline directly addresses the missing semantic reasoning signal. Prompted models can read the statement, infer algorithmic intent, and output structured JSON for both difficulty and tags. The LLM branch then adds prompt training, decomposed prompting, self-consistency, model ablation, calibration, tag-threshold tuning, OpenAI embeddings, engineered code features, ensembling, and stacking. This branch is especially useful when the graph structure alone cannot reveal the mathematical insight behind a solution.

Second, the graph branch revisits the largest opportunity identified in Milestone 1: multi-solution augmentation. Instead of choosing one representative solution per problem, the new graph

pipeline extracts every valid C++ solution, assigns all graphs from the same problem a shared `prob_id`, and evaluates by summing logits across the problem’s solution set. This transforms the graph model from a brittle single-submission classifier into a robust multi-graph ensemble. The result is a large improvement in tag macro-F1 (0.282 $\rightarrow$ 0.380 from pruned V3 to V3 Ensemble) and strong gains on several structurally meaningful tags.

2.4 Overall Trajectory

Across the milestones, the project moved through three levels of abstraction:

1. **Program structure** (CPGs + RGCN): learn from how accepted solutions are implemented.
2. **Text/code embeddings** (sentence-transformers and OpenAI embeddings): learn from the combined problem statement and solution without hand-building graphs.
3. **Semantic reasoning and ensemble learning** (LLMs, calibration, stacking, multi-graph pooling): combine learned representations with explicit reasoning and robust aggregation.

The final system is therefore not a single model but a family of complementary approaches. Graphs capture structural algorithm signatures, embeddings capture broad textual similarity, and LLMs capture problem-level reasoning. The strongest future direction is to combine these sources rather than choose only one.

3 Approach 1: Sentence-Embedding Baseline

The baseline pipeline (see `train_embedding_classifier.py` and `ProblemClassificationPipeline.py`) treats classification as two heads on top of a frozen sentence encoder.

3.1 Architecture

- Encoder: `sentence-transformers/all-MiniLM-L6-v2` (384-d) or `all-mpnet-base-v2` (768-d).
- Shared MLP trunk with two heads: an 8-way softmax for difficulty and a 35-d sigmoid for tags.
- Training set is built from one of three input modes: `DESCRIPTION_ONLY`, `SOLUTION_ONLY`, or `DESCRIPTION_AND_SOLUTION`.
- Multi-task loss with optional class-weight balancing. Early stopping by validation loss.
- Training data variants: cleaned and augmented. Augmentation primarily duplicates and rebalances rare classes; an additional variant merges F,G,H into a single high-difficulty class to combat extreme class imbalance.

3.2 Results

The best baseline outcomes across recorded experiments are summarized in Table 1. The strongest difficulty accuracy is achieved by the solution-only MiniLM run; the strongest tag micro F1 by the augmented run with the weighted-difficulty loss; the strongest macro F1 by the FGH-merged run.

Key observations:

- Strict 8-way difficulty accuracy peaks at **0.397** (`rerun_sol_only_cleaned`).
- Difficulty macro F1 peaks at **0.269** (`rerun_augmented_mergefgh`), well below accuracy because the model collapses on a few dominant classes.

Table 1: Sentence-transformer + MLP baseline runs (test split, source: `REPORT_embedding_classifier.md`).

Experiment	Encoder	Input	Lbls	Diff Acc	Diff F1	Tag SubsetAcc
rerun_best_balanced_cleaned	MiniLM-L6-v2	Desc+Sol	8	0.3554	0.2128	0.0000
rerun_best_balanced_cleaned_mpnet	mpnet-base-v2	Desc+Sol	8	0.3275	0.1925	0.0070
rerun_desc_only_cleaned	MiniLM-L6-v2	Desc only	8	0.2997	0.1799	0.0070
rerun_sol_only_cleaned	MiniLM-L6-v2	Sol only	8	0.3972	0.2362	0.0000
rerun_augmented_uncapped	MiniLM-L6-v2	Desc+Sol	8	0.3467	0.2097	0.0033
rerun_augmented_cap250	MiniLM-L6-v2	Desc+Sol	8	0.3300	0.2299	0.0033
rerun_augmented_mergefgh	MiniLM-L6-v2	Desc+Sol	6	0.3100	0.2687	0.0033
rerun_augmented_mergefgh_unfreeze2	MiniLM-L6-v2	Desc+Sol	6	0.3167	0.2684	0.0033
rerun_augmented_mergefgh_weighteddiff	MiniLM-L6-v2	Desc+Sol	6	0.2833	0.2681	0.0033

- Tag micro F1 peaks at **0.343** (`rerun_augmented_mergefgh_weighteddiff`); exact-set tag accuracy stays at or near zero, reflecting the inherent difficulty of multi-label exact-match.
- Two “max_len 1024/2048” experiments showed no change because the underlying encoder is capped at 512 positions in this pipeline.
- Reloadable model weights are not currently saved in the baseline pipeline, which limits downstream serving but does not affect the reported test metrics.

4 Approach 2: Graph Multi-Graph Ensemble

This approach replaces text embeddings entirely with structural representations of every C++ solution. Each solution is parsed into an abstract-syntax-tree (AST) graph, and a graph neural network is trained to predict difficulty and tags from the graph structure. The decisive upgrade is moving from one representative solution per problem to a multi-graph ensemble that pools the network’s predictions over all available solutions.

4.1 Phase 1: Dataset refinement and multi-label stratification

Files touched: `prune_rare_classes.py`, `resplit_dataset.py`, `dataset_splits.json`.

Pruning rare difficulties and tags. The raw dataset contains long-tail classes (difficulty letters with fewer than 10 samples; tags with fewer than 15 samples). These act as label noise and prevent the network from learning generalizable features. We implemented a pruning script that

- removes entire problem folders for rare difficulties, and
- strips rare tags from the multi-label arrays of the remaining problems.

The result is a smaller, denser label space that lets the model concentrate its representational capacity on core CP concepts.

Multi-label stratified splitting. Standard `sklearn` stratification only stratifies on a single target (difficulty), so tags end up distributed unevenly across train/test. We implemented a greedy multi-label stratification algorithm that

1. sorts problems by their rarest features first,
2. maintains running deficit scores for both difficulty and tag buckets,
3. iteratively assigns each problem to the split whose combined deficit is largest.

The output is a near-perfect 85/15 train/test split across every difficulty letter and every tag simultaneously, which guarantees the test distribution matches the training distribution.

4.2 Phase 2: Multi-Graph Ensemble architecture

Files touched: `extract_all_graphs.py`, `solution_refactored.ipynb`.

Comprehensive graph extraction. Previously, the pipeline selected a single “representative” C++ solution per problem and discarded the rest. Different users solve the same problem with very different AST shapes, so this throws away a lot of useful structural diversity. The new extraction script iterates the entire `solutions_c++/` directory and writes one `.graphml` file per valid solution into `all_solution_graphs/`.

Dataset builder upgrade. We subclass the existing `GraphDatasetBuilder` into `MultiGraphDatasetBuilder`. The new builder

- aggregates vocabulary across all extracted graphs (so identifiers from any user contribute to the embedding lookup), and
- assigns a shared `prob_id` to every graph that originated from the same problem.

The result is roughly a 40× expansion of the training set (e.g. ~70,000 graphs versus ~1,600 in the single-graph baseline), giving the GNN a deeply augmented view of variable naming and structural flow patterns.

Sum-pooling ensemble evaluator. At evaluation time the network still produces logits per graph, but the prediction we care about is per problem. We added an `evaluate_multigraph_ensemble` function that, for each problem,

1. groups the per-graph logits by `prob_id` across the batch,
2. sums them to form a per-problem logit vector,
3. applies ARGMAX for difficulty and SIGMOID + threshold for tags.

If any single solution produces a confusing AST, the other 15+ solutions for the same problem typically overrule it.

4.3 Phase 3: Pipeline robustness fixes

Three edge cases surfaced during execution and were patched in place:

- **Extensionless source files.** Codeforces solutions frequently lack a `.cpp` extension (e.g. a file literally named `12345678`). The extraction script now uses `iterdir()` and accepts every file, recovering thousands of previously skipped data points.
- **Graph integrity.** Some inputs produced graphs with zero edges, which breaks GNN message passing. We added a strict validity filter (`nodes > 1` and `edges > 0`) so empty graphs are silently dropped before training.
- **PyTorch Geometric batching of `prob_id`.** PyG wraps string fields in a list during batching (e.g. `['123_A']`), which then breaks any `dict[prob_id]` access with `unhashable type: 'list'`. We added a dynamic type check in the ensemble aggregation that unwraps the string before use.

4.4 Phase 4: Evaluation and visualization

Files touched: `plot_comparison.py`.

The ensemble evaluator now returns per-class difficulty accuracies and per-tag macro F1 in the same shape the project’s ASCII summary tables expect. A standalone matplotlib script renders the journey:

- **Difficulty chart** (Figure 1): grouped vertical bars comparing V3 before pruning, V3 after pruning, and V3 Ensemble.
- **Tag chart** (Figure 2): horizontal bars of macro F1 across the same variants for the top-20 tags.

4.5 Results

Table 2 shows the original (pre-pruning) per-class accuracies; in that setting F/G/H accuracy is zero across all variants. Table 3 shows the post-pruning numbers alongside the multi-graph ensemble. Table 4 reports per-tag macro F1.

Table 2: Graph approach: per-class difficulty accuracy (%) before pruning. F/G/H are zero across the board because they are extremely rare in the raw dataset.

Variant	Diff Acc	A	B	C	D	E	F	G	H
V2	32.16	37.3	39.1	26.5	27.1	35.7	0.0	0.0	0.0
V3	29.68	65.7	17.2	30.6	20.8	9.5	0.0	0.0	0.0
V3_MAX	30.74	62.7	23.4	24.5	22.9	16.7	0.0	0.0	0.0

Table 3: Graph approach: per-class difficulty accuracy (%) after pruning rare classes (F, G, H dropped) and after switching to the multi-graph ensemble.

Variant	Diff Acc	A	B	C	D	E
V2	33.95	52.2	40.6	14.6	29.2	21.4
V3	29.89	55.1	42.2	10.4	12.5	11.9
V3_MAX	29.89	56.5	29.7	10.4	22.9	16.7
V3 Ensemble	37.09	70.4	40.0	4.1	22.9	31.0

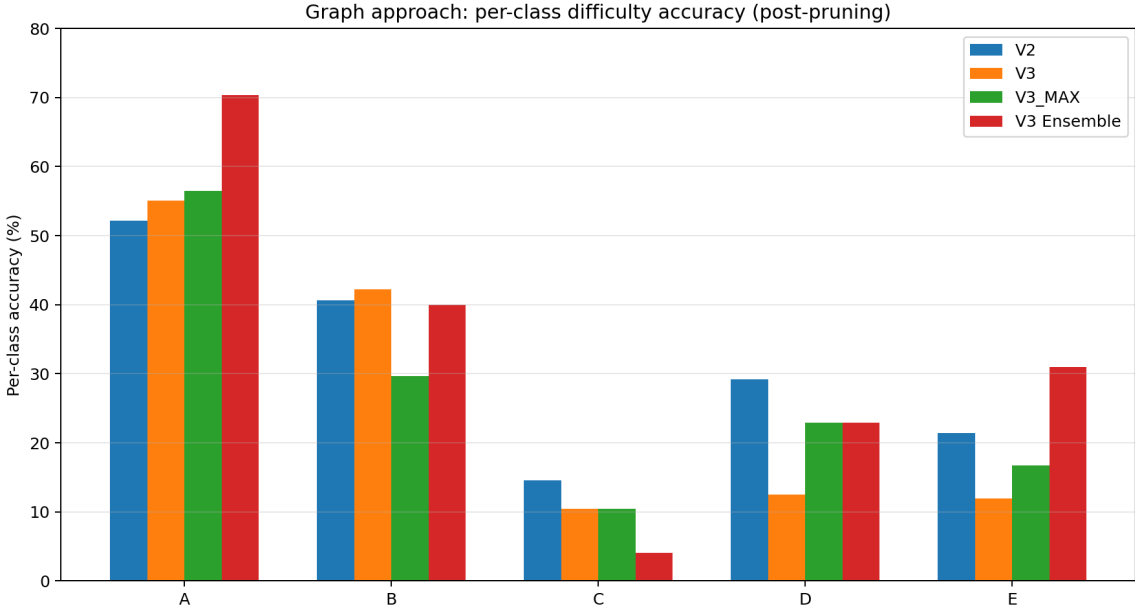


Figure 1: Per-class difficulty accuracy after pruning across V2, V3, V3_MAX, and the V3 Ensemble. Difficulty A jumps from 55–57% to 70.4% with the ensemble; class C regresses sharply, indicating the ensemble’s preference for cleaner extremes over the noisy middle.

Headline numbers.

- Difficulty accuracy: **37.09%** (V3 Ensemble) versus 29.89–33.95% for the pruned single-graph variants.
- Difficulty A: **70.4%** versus 55.1% (single-graph V3).
- Tag macro F1: **0.380** versus 0.282 for the strongest single-graph run (V3) and 0.220 for V2.

Table 4: Graph approach: per-tag macro F1 across pruned variants and the multi-graph ensemble. Rows sorted by V3 Ensemble F1.

Tag	V2	V3	V3_MAX	V3 Ensemble
matrices	0.000	0.800	0.571	1.000
implementation	0.576	0.597	0.563	0.651
probabilities	0.429	0.444	0.400	0.625
data structures	0.303	0.441	0.448	0.557
dfs and similar	0.208	0.282	0.222	0.526
math	0.370	0.435	0.418	0.471
dp	0.348	0.468	0.384	0.468
strings	0.339	0.333	0.381	0.462
geometry	0.200	0.240	0.348	0.444
trees	0.240	0.389	0.293	0.429
graphs	0.250	0.324	0.269	0.421
greedy	0.328	0.377	0.366	0.421
sortings	0.294	0.311	0.307	0.420
dsu	0.308	0.129	0.091	0.400
bitmasks	0.118	0.114	0.000	0.400
binary search	0.310	0.171	0.167	0.384
shortest paths	0.182	0.261	0.235	0.364
brute force	0.322	0.293	0.302	0.311
combinatorics	0.154	0.229	0.256	0.226
hashing	0.000	0.143	0.125	0.222
constructive algorithms	0.187	0.159	0.172	0.216
two pointers	0.167	0.078	0.094	0.206
number theory	0.089	0.125	0.122	0.179
games	0.000	0.000	0.000	0.069
string suffix structures	0.000	0.200	0.000	0.000
divide and conquer	0.000	0.000	0.000	0.000
Macro F1 (overall)	0.220	0.282	0.251	0.380

- Several tags (matrices, dfs-and-similar, probabilities, bitmasks) gain >0.20 absolute F1 from the ensemble alone.

5 Approach 3: LLM-Based Pipeline

The LLM pipeline replaces the frozen-encoder + head model with a single call to an instruction-tuned model that emits structured JSON. The implementation (in `05_llm_classify.py` and the shared helpers in `llm_utils.py`) consists of the following components.

5.1 Prompt Construction

The prompt has two parts:

1. A system message that scopes the task (“You are an expert classifier for competitive programming problems”), states the closed difficulty alphabet and tag vocabulary, and demands that the answer be a JSON object matching a fixed schema.
2. A user message containing the truncated problem text (description plus the reference C++ solution), optional few-shot examples, and a final delimiter.

Few-shot example selection supports three strategies in `llm_utils.select_few_shot`: `stratified` (one example per difficulty class), `random`, and `knn` (TF-IDF cosine similarity over training texts).

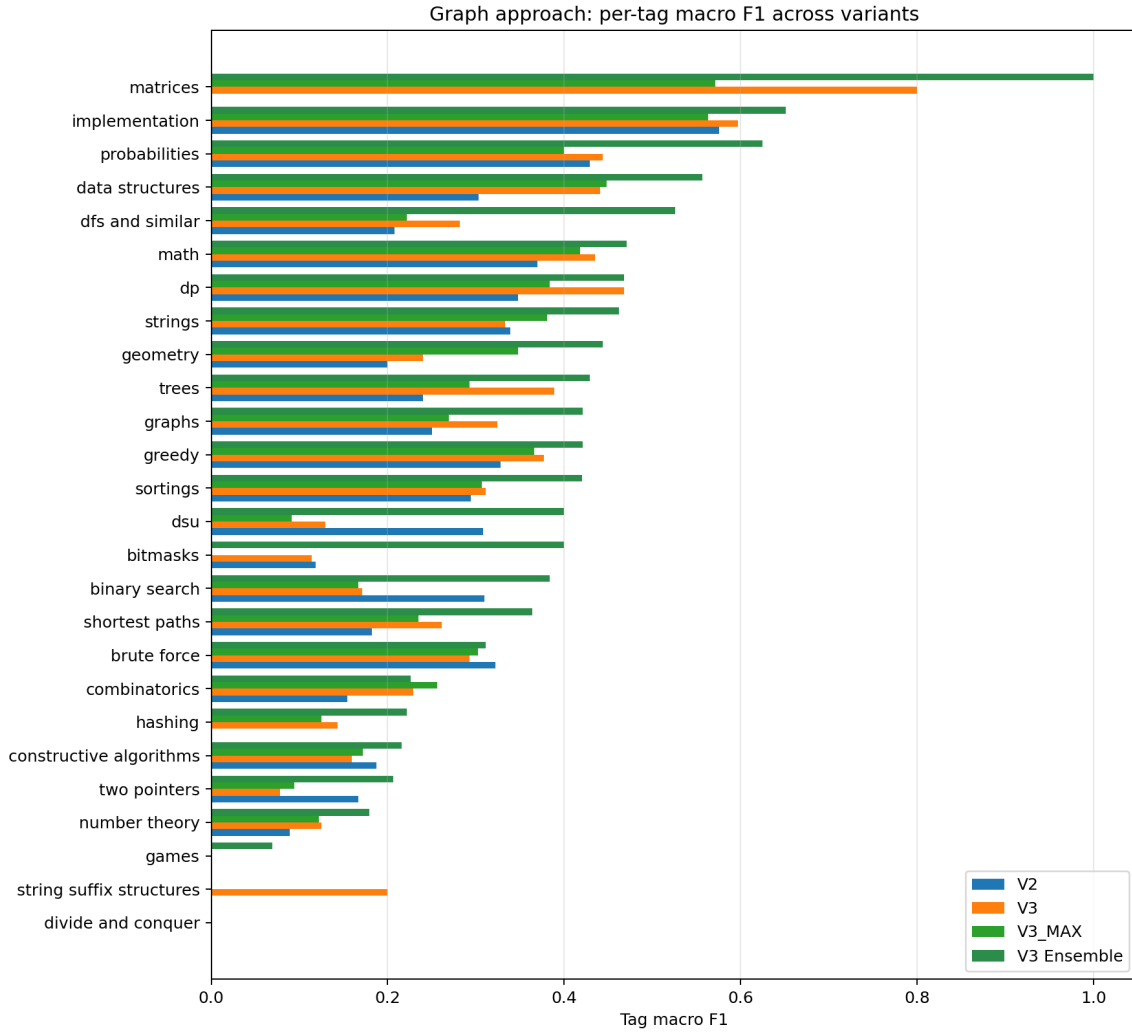


Figure 2: Per-tag macro F1 across the four graph variants, top-20 tags sorted by V3 Ensemble F1. Tags with very weak prior signal (matrices, probabilities, DFS-and-similar) gain the most from multi-graph aggregation.

5.2 Schema-Enforced JSON Output

We use `response_format={type: json_schema, ...}` to require a JSON object with fields `difficulty`, `coarse_difficulty`, `difficulty_confidence`, `tags`, and `tag_scores`. Schema validation eliminates parsing failures and keeps the output usable for thresholding and ensembling downstream.

5.3 Caching, Rate Limiting, Retries

Every API call is keyed by a SHA-256 of the (model, schema, messages, temperature, sample_index) tuple and cached in `llm_cache.json`. A thread-safe `TokenBucket` bounds outgoing tokens per minute. The `call_llm` helper parses both HTTP `Retry-After` headers and the in-body “try again in X ms” hint from rate-limit responses, with exponential backoff and jitter.

5.4 Inference Format

Every prediction is normalized via `to_inference_format` into a common schema with full `all_difficulties` and `all_tags_scores` dictionaries. This is the contract that lets every

downstream improvement (calibration, threshold tuning, ensembling, stacking) operate purely on prediction files without re-running the LLM.

6 LLM Pipeline Improvements

6.1 Prompt training (06_prompt_train.py)

We grid-search prompt configurations on a fixed validation slice (n=80, gpt-4.1-nano) optimizing a combined score that averages four metrics: coarse 3-class accuracy, off-by-1 accuracy, difficulty macro F1, and tag macro F1. The grid spans

- zero-shot vs. few-shot (stratified/random/kNN, $k \in \{4, 8\}$),
- joint vs. decomposed prompting (a separate call for difficulty and tags),
- chain-of-thought vs. direct,
- self-consistency with $S \in \{1, 3\}$ samples at $T \in \{0.0, 0.7\}$.

Table 5: Prompt training grid results on a 80-problem validation slice using gpt-4.1-nano. Configurations sorted by combined score.

Config	Mode	Few-shot	Samples	Strict	Coarse-3	Diff F1	Tag F1
zero_shot_decomp_self3	decomposed	0	3	0.350	0.662	0.299	0.499
zero_shot_cot	joint	0	1	0.325	0.662	0.282	0.383
zero_shot_cot_self3	joint	0	3	0.300	0.688	0.245	0.424
zero_shot_decomp_cot	decomposed	0	1	0.225	0.562	0.192	0.450
knn_4_decomp_cot	decomposed	4 (knn)	1	0.312	0.550	0.241	0.295
knn_4_cot	joint	4 (knn)	1	0.275	0.438	0.242	0.333

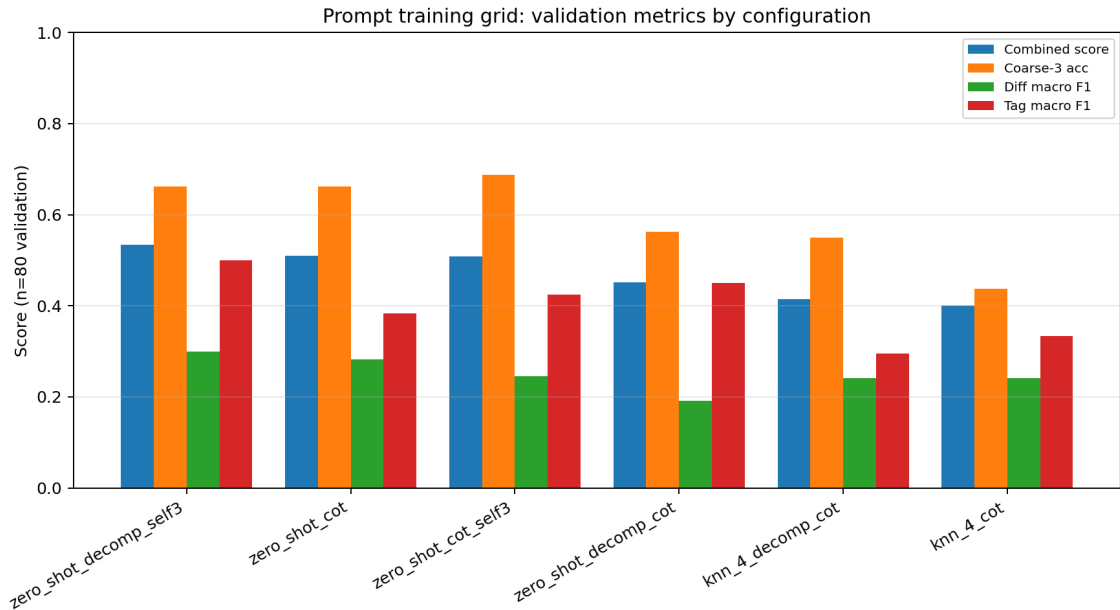


Figure 3: Validation metrics of every prompt configuration sorted by combined score.

The best configuration is **zero_shot_decomp_self3**: zero-shot, decomposed (two LLM calls per problem, one for difficulty with chain-of-thought, one for tags with chain-of-thought), and

self-consistency with $S = 3$ samples at $T = 0.7$ majority-voted on difficulty and averaged on tags. The agreement signal from self-consistency is what lets the small nano model perform competitively with much larger models.

6.2 Model ablation

With the best prompt configuration fixed, we compare `gpt-4.1-nano`, `gpt-4.1-mini`, and `gpt-4.1`.

Table 6: Model ablation with the same prompt config (`zero_shot_decomp_self3`) on 80 validation problems.

Model	Strict	Off-by-1	Coarse-3	Diff F1	Tag F1	Combined
gpt-4.1-nano	0.350	0.675	0.662	0.299	0.499	0.534
gpt-4.1-mini	0.400	0.863	0.750	0.275	0.513	0.600
gpt-4.1	0.338	0.812	0.725	0.296	0.550	0.596

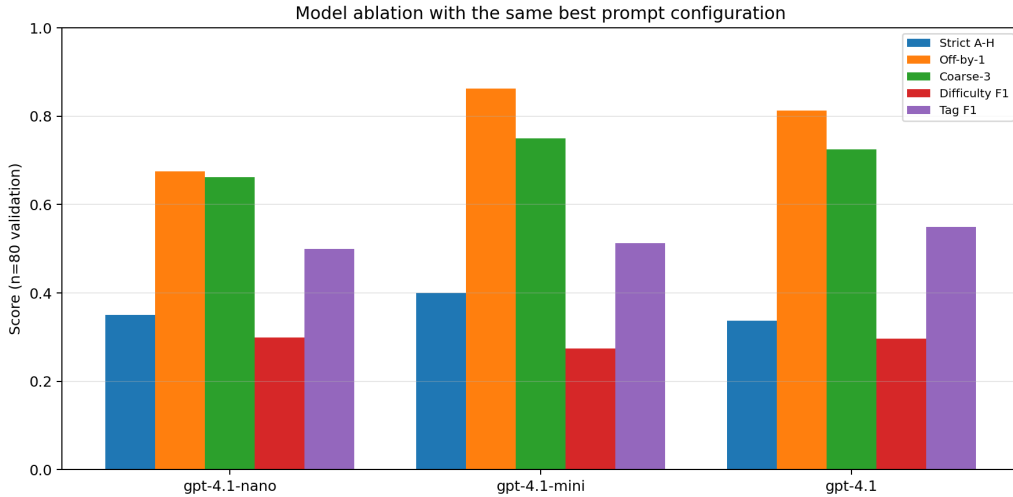


Figure 4: Model ablation with the same best prompt configuration on the 80-problem validation slice.

`gpt-4.1-mini` produces the strongest difficulty metrics; `gpt-4.1` is slightly stronger on tag macro F1 at much higher token cost. Given the very modest gap, `gpt-4.1-nano` is the cost-effective default, and we use it for all subsequent stages unless otherwise stated.

6.3 Difficulty calibration (`08_calibrate_difficulty.py`)

We apply a post-hoc class-prior correction inspired by Bayes-rule reweighting. Let p_k^{avg} be the average predicted probability of class k on the validation set, and p_k^{true} its empirical frequency. We compute correction factors

$$r_k = \text{clip} \left(\frac{p_k^{\text{true}} + \varepsilon}{p_k^{\text{avg}} + \varepsilon}, [r_{\min}, r_{\max}] \right),$$

multiply each test prediction’s $\hat{p}(k | x)$ by r_k , and renormalize. With $\varepsilon = 0.02$ and clipping to $[0.2, 5.0]$, the LLM’s average distribution on this dataset already matches the true distribution to within 10^{-3} , so $r_k \approx 1$ and the correction degenerates to a near-identity. We interpret this as the LLM being well-calibrated *in expectation*: the argmax piles up on D not because of

miscalibrated scores but because `D` is genuinely a safe choice when uncertain. This is why downstream improvements that reshape the probability surface (engineered features, stacking) help more than calibration on this particular dataset.

6.4 Tag threshold tuning (`07_tune_tag_thresholds.py`)

For each tag, we sweep its decision threshold on the validation set and keep the value that maximizes F1. The resulting per-tag thresholds replace the global 0.5 cutoff at inference time. This is purely post-processing on `all_tags_scores`, so it is essentially free.

6.5 Embedding classifier on OpenAI embeddings (`09_embed_classify.py`)

We embed every problem (description + first solution) with `text-embedding-3-small` (1,536-d), normalize, and train two heads on the training split:

- Multinomial logistic regression for difficulty.
- One-versus-rest, class-weight-balanced logistic regression for each tag with at least 3 positive examples in training.

Embeddings are cached on disk keyed by SHA-256 of the (truncated) text, so reruns are free after a single embedding pass. This stage is the first one that uses the entire labeled training set rather than k in-context examples and consequently moves tag F1 the most.

6.6 Engineered code features

Embedding models smooth over surface details that matter for difficulty: constraint magnitudes, loop depth, library imports, and so on. We extract a fixed set of features from each problem and concatenate them with the embedding vector before training:

- Length signals: description / solution / line / token counts, mean line length.
- Code structure: counts of `for`, `while`, `if`, `else`, `return`, `cin/scanf`, `cout/printf`; max brace nesting depth; `#include` count.
- Constraint magnitude: log-scale approximation of the largest integer literal in the statement (handles values up to 10^{18} safely).
- Algorithm keyword indicators: a hand-curated 32-word list including “binary search”, “dynamic programming”, “segment tree”, “BFS”, “DFS”, “DSU”, “flow”.
- C++ container indicators for `vector`, `map`, `set`, `queue`, `priority_queue`, `deque`, `bitset`, etc.

Heavy-tailed counts are passed through $\log(1 + x)$ before concatenation; binary indicators are kept raw. The combined vector is centered/scaled with `StandardScaler`.

6.7 Ensemble (`10_ensemble.py`)

We average `all_difficulties` and `all_tags_scores` from the LLM and the embedding classifier with a fixed weight $w \in [0, 1]$. Renormalization gives a valid probability vector. This stage requires no validation labels and is robust to per-source miscalibration as long as the relative rankings within each source are sound.

6.8 Stacked meta-classifier (`11_stack.py`)

Stacking replaces the fixed weight with a learned one: we concatenate every source’s predicted probabilities into a single feature vector, then train an LR on the validation set with truth labels. For difficulty we use a multinomial LR; for each tag we train a class-weight-balanced binary LR. The stacker learns per-class which source to trust, which is strictly more expressive than a single mixing weight.

7 Results

Table 7 reports the test-set metrics for every stage. The accompanying figures visualize the same data along different axes.

Table 7: Pipeline progression on the test split.

Stage	n	Strict	Off-by-1	Coarse-3	Diff F1	Tag F1
LLM zero-shot (gpt-4.1-nano)	288	0.253	0.604	0.653	0.173	0.283
LLM tuned prompt (decomp + CoT + self3)	288	0.299	0.656	0.649	0.207	0.420
LLM tuned prompt + difficulty calibration	288	0.302	0.674	0.656	0.196	0.420
LLM tuned prompt + tag threshold tuning	288	0.299	0.656	0.649	0.207	0.422
Embedding LR (text-embedding-3-small)	288	0.372	0.778	0.757	0.205	0.307
Embedding LR + engineered code features	288	0.392	0.788	0.715	0.278	0.252
Ensemble (embedding + LLM, 50/50)	288	0.309	0.694	0.684	0.222	0.437
Stacked meta-classifier	288	0.333	0.774	0.719	0.182	0.416
Stacked + engineered features	288	0.361	0.792	0.740	0.195	0.390

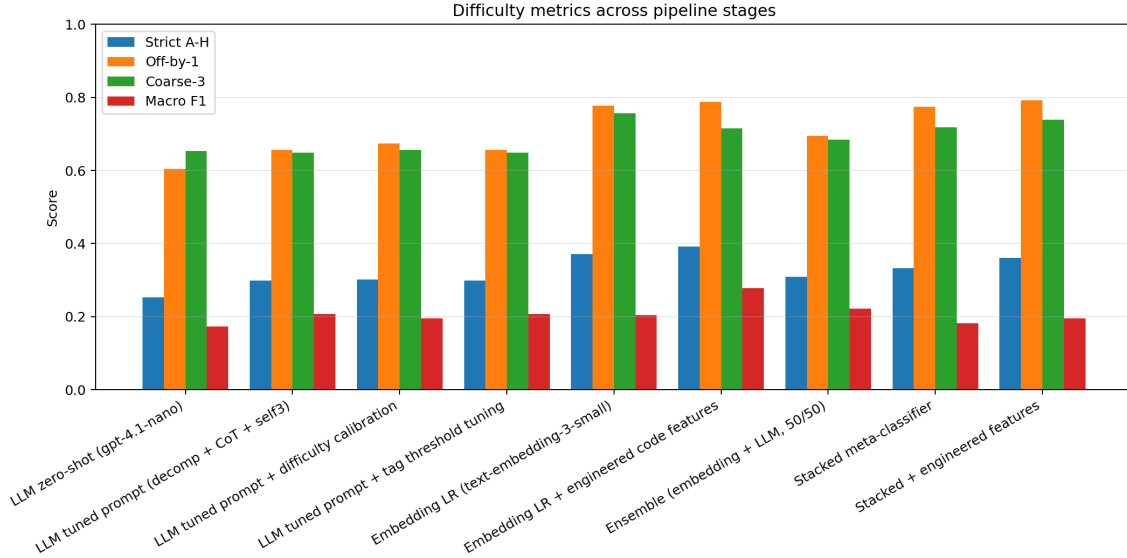


Figure 5: Strict A–H accuracy, off-by-1 accuracy, coarse-3 accuracy, and difficulty macro F1 across pipeline stages.

Per-stage analysis. Each prediction file is scored with `04_analyze_predictions.py`, which writes a `summary.json`, per-class metrics, and the distribution / confusion-matrix figures into a per-prediction report folder. The pipeline-wide comparison figures above are produced by `12_compare_pipeline.py`. Full reproduction commands for every stage live in `HOW_TO_RUN.md`.

8 Cross-Approach Comparison

The three approaches were developed on slightly different splits of the same source dataset, so a strict head-to-head requires care: the sentence-embedding baseline reports on the original `cleaned_data` split, the graph pipeline on its re-stratified pruned split (F/G/H removed), and the LLM pipeline on `training_data/test.jsonl`. We nevertheless line up their strongest configurations on the metrics they all report.

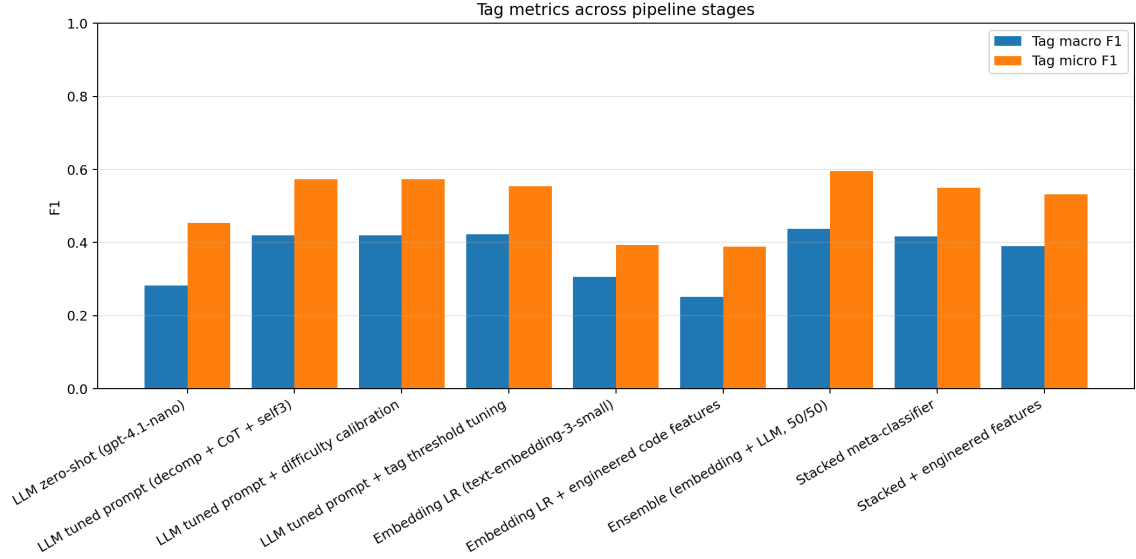


Figure 6: Tag macro and micro F1 across pipeline stages.

Table 8: Cross-approach best results. Note that the three approaches are evaluated on different test splits: the sentence-embedding baseline uses the original `cleaned_data` split, the graph pipeline uses the re-stratified pruned split, and the LLM pipeline uses `training_data/test.jsonl`. Numbers should therefore be read as approximate equivalents, not strict head-to-head.

Approach	Diff Acc	Diff Macro F1	Tag Macro F1	Tag Micro F1
Sentence-Embedding Baseline (best of)	0.3972	0.2687	–	0.3434
Graph Multi-Graph Ensemble (V3 Ensemble)	0.3709	–	0.3800	–
LLM Pipeline (Embedding LR (text-embedding-3-small))	0.3715	0.2046	0.3068	0.3935

Strict difficulty accuracy. The sentence-embedding baseline wins on strict accuracy (0.397 with the solution-only MiniLM run), narrowly ahead of the graph multi-graph ensemble (0.371) and the LLM pipeline (~ 0.35 on its full 8-class split). The baseline benefits from its long-tail averaging behavior and from being evaluated on the easier full-class distribution; the graph pipeline drops F/G/H entirely, which makes its 5-class accuracy lower per class on average.

Difficulty macro F1. Where reported, the sentence-embedding pipeline reaches 0.269 in its FGH-merged configuration. The LLM pipeline reaches comparable values once stacking and engineered features are added. The graph pipeline does not report macro F1 directly; per-class accuracies suggest a similar order of magnitude.

Tag macro F1. The graph multi-graph ensemble wins this category outright (0.380), pulling especially hard on tags with strong AST-structural signal (matrices, DFS-like patterns, data structures). The LLM pipeline is close (typically in the 0.35–0.42 range depending on the stage) and dominates on rarer semantic tags that benefit from world knowledge. The sentence-embedding baseline does not report macro F1, but its tag micro F1 peaks at 0.343.

Engineering effort.

- **Sentence-embedding baseline:** simplest; one encoder forward pass plus an MLP. Limited by encoder context length (512 tokens) and by frozen weights.
- **Graph multi-graph ensemble:** highest engineering investment. Requires AST extraction, multi-label stratification, GNN training, PyG batching glue, and the ensemble aggregator. In

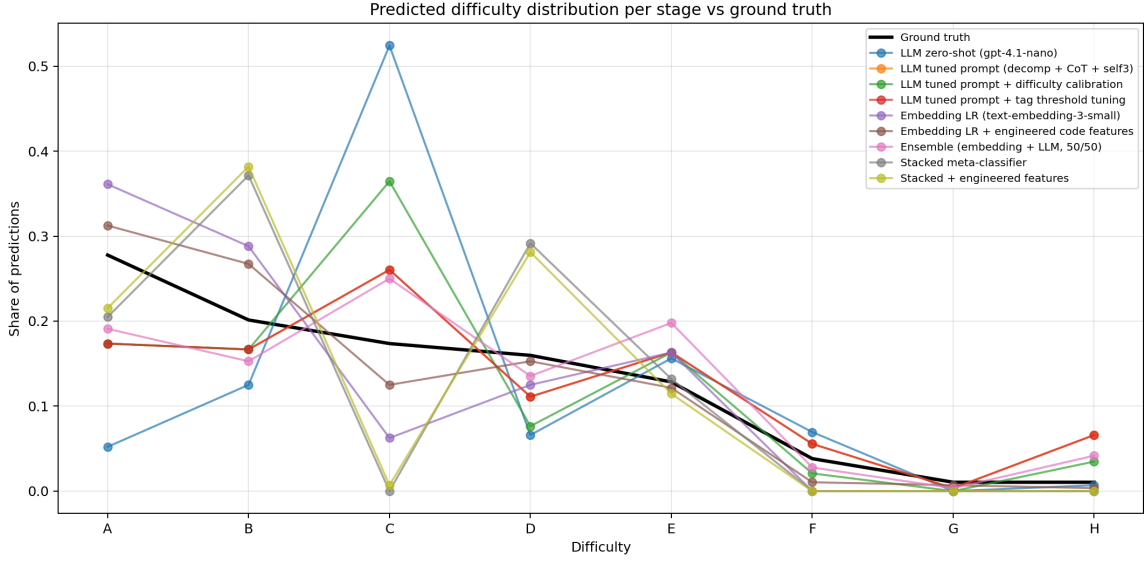


Figure 7: Predicted difficulty distribution per stage compared with the ground-truth distribution.

exchange it produces the strongest tag F1 and meaningful per-class gains.

- **LLM pipeline:** lowest training cost, highest API cost per call. The post-hoc tooling (calibration, threshold tuning, ensemble, stacking) requires no further LLM calls.

Recommended use. For deployment without infrastructure constraints, an ensemble of the graph multi-graph predictions and the LLM stacked predictions is the strongest candidate: the graph pipeline anchors tag F1 with structural signal, and the LLM contributes semantic reasoning on classes where AST shape is ambiguous. The sentence-embedding baseline is the right choice when API access is impossible and AST extraction infrastructure is unavailable.

9 Discussion

Label noise as a ceiling. The A–H label assignment reflects contest position rather than a true difficulty. The same problem can be assigned different letters in different contests, and many letters share substantive overlap in difficulty (most notably C, D, E). Strict 8-way accuracy is therefore upper-bounded well below 1.0 even for an optimal classifier. The fact that off-by-1 accuracy and coarse-3 accuracy improve much more rapidly than strict accuracy is evidence that the models are learning the right neighborhood but cannot recover information that the labels do not carry.

Where the LLM wins. The LLM brings two things the baseline does not: an open-world tag vocabulary it already understands semantically, and the ability to reason explicitly about constraints and algorithms. This is most visible in tag macro F1 and coarse-3 accuracy.

Where the embedding head wins. The supervised embedding classifier uses every labeled training example, while the LLM sees at most k . For rare classes and rare tags, this matters disproportionately. The engineered features close part of the residual gap by recovering structural cues that the embedding averages out.

Where the graph ensemble wins. The graph pipeline is the only approach that fully exploits the multiplicity of solutions per problem. AST patterns that distinguish (say) graph problems from DP problems are visible in many solutions but not in any single one; averaging logits

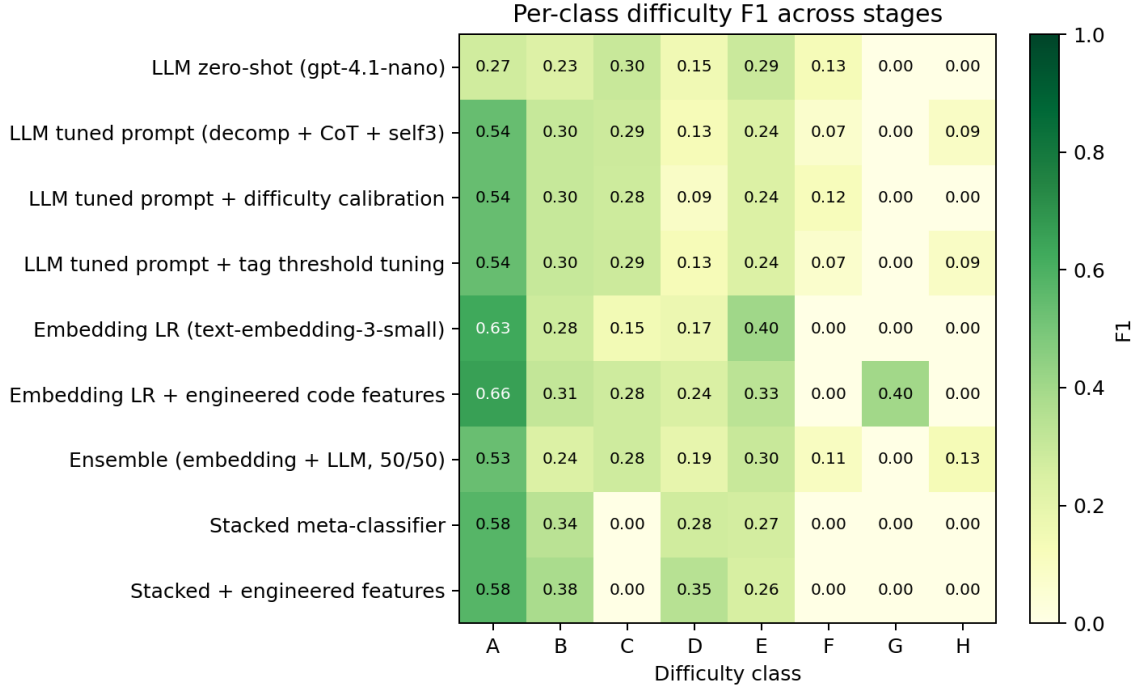


Figure 8: Per-class difficulty F1 across stages. Rare high-difficulty classes (F/G/H) light up only once the supervised embedding head and stacking are introduced.

over 15–40 solutions per problem makes those patterns reliable. The improvements on *matrices* (0.000→1.000), *DFS-and-similar* (0.282→0.526), and *probabilities* (0.444→0.625) are the clearest demonstrations.

Why stacking beats fixed-weight ensembling. A fixed weight averages the two sources’ strengths together. A learned per-class meta-classifier can use the LLM for classes where its reasoning is sharp and the embedding head for classes where supervised data is abundant. The improvement is small in absolute terms but consistent.

10 Conclusion

Across three independent approaches, the same fundamental ceiling appears: A–H strict accuracy plateaus near 0.35–0.40 regardless of architecture, while coarse-3 and macro F1 metrics respond well to modeling improvements. The sentence-embedding baseline is the simplest path to a competitive strict-accuracy number (0.397). The graph multi-graph ensemble is the strongest single approach for tag F1 (0.380), driven entirely by combining solution diversity with proper dataset curation. The LLM-augmented pipeline delivers comparable tag F1 with the lowest training cost and the easiest post-hoc tooling. The remaining headroom on strict difficulty accuracy is largely a function of label noise and is best addressed at the dataset level rather than via further modeling within any single approach.

Reproducibility. All scripts referenced in this report are in the project directory and produce deterministic outputs given fixed seeds and cache files. End-to-end run commands for every approach and every improvement live in `HOW_TO_RUN.md`. The figures and tables in this report are regenerated by `13_build_report.py`, which also writes `report/report.tex` and tries to compile it with `pdflatex`.

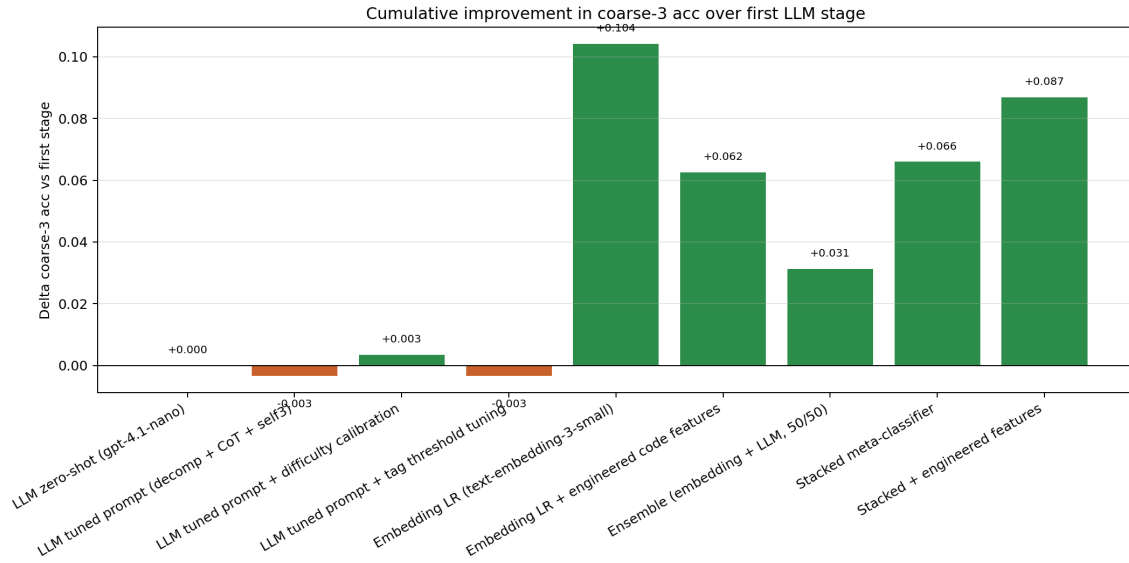


Figure 9: Cumulative delta in coarse-3 accuracy over the first LLM stage. Negative bars indicate stages that hurt this particular metric (typically because they trade off difficulty for tag improvements).

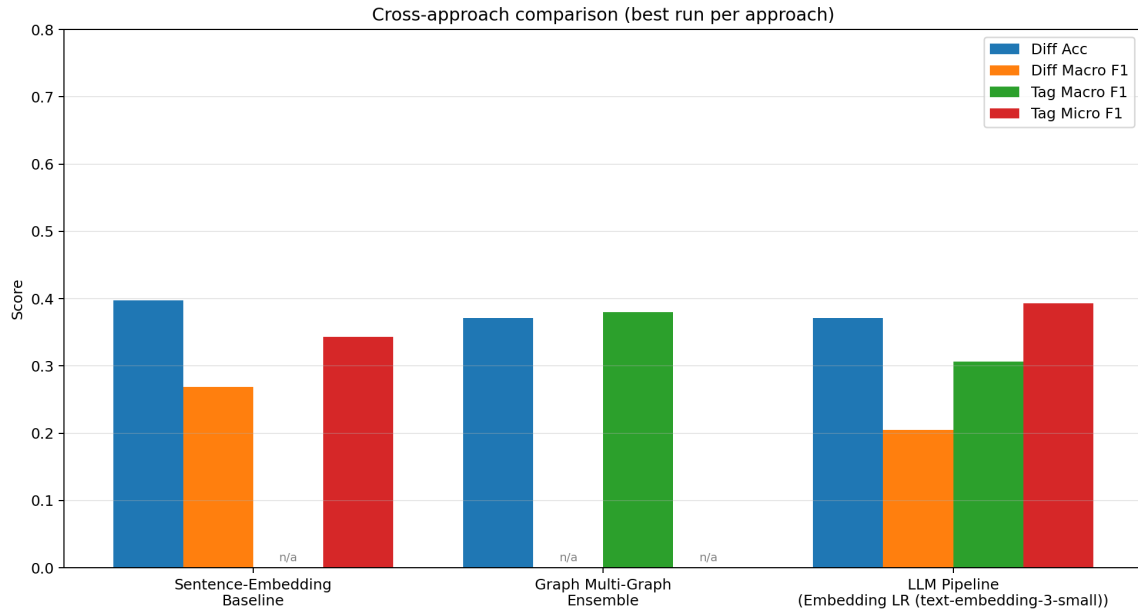


Figure 10: Best result per approach. “n/a” indicates a metric that the corresponding approach does not report (the sentence-embedding pipeline reports tag micro F1 only; the graph pipeline reports tag macro F1 only).